

University of Southern Denmark
The engineering programmes in Sønderborg

Advanced Object-Oriented Programming

Summer EXAMINATION

Thursday, 18th August 2025

Problem 1 starts on the next page!

General Information

In general, for all tasks you can only use C# and the following libraries:

The .Net System library, Avalonia and CommunityToolkit.MVVM.

To avoid excessive subfolders and ensure that all your work is found and graded, only submit the requested files for each problem in a **flat folder structure**—that means a single-level folder containing all the required files. **Do not** submit a separate folder for each problem, and make sure there are no unnecessary folders like /bin/, /obj/, etc.

Your final submission folder should only contain 6 files and look **exactly** like this:

```
/AOP_Rexam_Submission/  
|— Problem_1_Submission.txt  
|— Problem_2_MainWindowViewModel.cs  
|— Problem_2_MainWindow.axaml  
|— Problem_3_MainWindowViewModel.cs  
|— Problem_4_Program.cs  
|— Problem_4_Query_Results.json
```

Problem 1 – SOLID Principles (25 Points)

In the folder Problem_1_SOLID\FamilyMealPlanner\ you can find a fully functional C# console project called *FamilyMealPlanner*. Analyze the code to answer the following questions. Write your answers in the provided text file: Problem_1_Submission.txt

For each of the 5 SOLID principles:

- **State whether the principle is present** in the provided code.
(5 points – 1 point per principle)
- **If present, briefly describe how and where it appears** in the code.
(10 points – 2 points per principle)
Provide at least one example from the code.
- **If present, briefly explain the general purpose of the principle** and how it applies specifically in this code.
(10 points – 2 points per principle)

Problem 2 – UI (30 Points)

In the folder `Problem_2_UI\FamilyMealPlannerUI\`, you will find a basic Avalonia project *FamilyMealPlannerUI* that provides a user interface for the Family Meal Planner program from Problem 1. All core functionality has already been injected into this project so all that is missing is the UI and corresponding UI Logic—your task is to implement the UI and corresponding UI logic.

The goal of this application is to allow a user to: Generate and display a **meal plan for a whole week** and summarize the **shopping list** based on the meal plan.

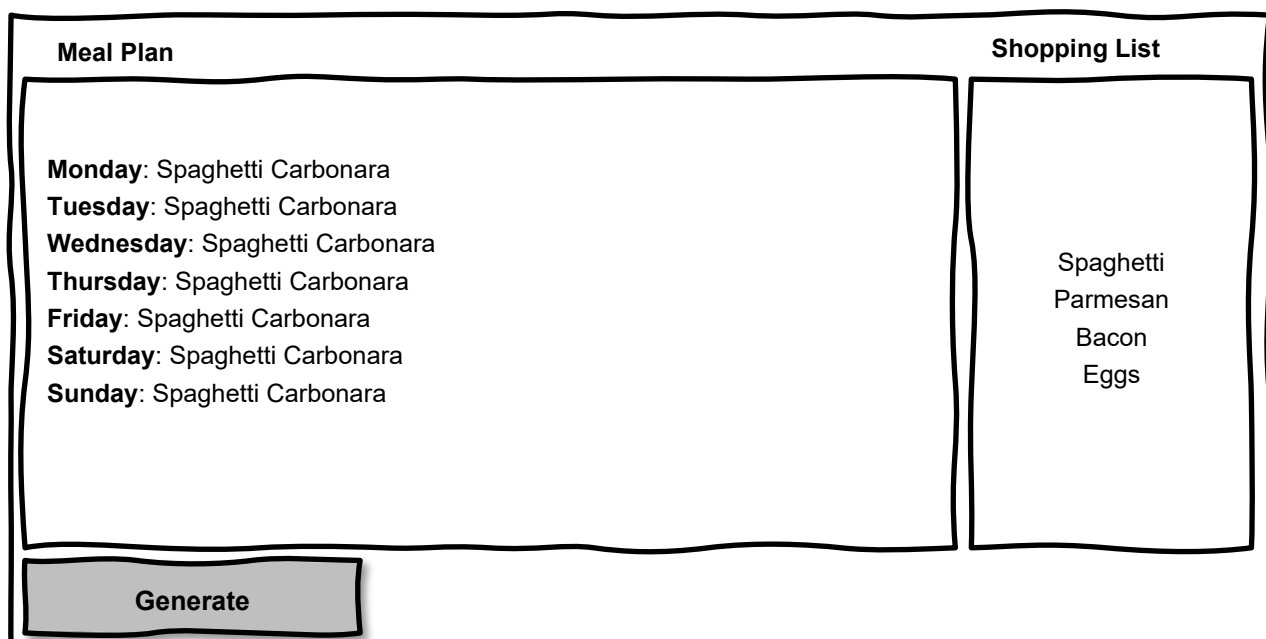
Follow the subtasks below and implement the required UI and functionality.

Strictly follow the MVVM pattern, so only hand in the following **two files** for this problem. Any additional files will not be graded. Include only these files in your submission folder:

- `Problem_2_UI/FamilyMealPlannerUI/ViewModels/Problem_2_MainWindowViewModel.cs`
- `Problem_2_UI/FamilyMealPlannerUI/Views/Problem_2_MainWindow.axaml`

2.1. Layout (10 Points)

Using the provided sketch as a reference, modify the `MainWindow.axaml` file so that the UI layout closely resembles the sketch. **No functionality is required for this subtask**—focus solely on the visual structure. The window should contain **two ListBox elements**: **Left ListBox**: Will display the generated recipes for each weekday. **Right ListBox**: Will display the combined shopping list of all needed ingredients. **Below the ListBoxes**, there should be a **Button** labeled something like “Generate Meal Plan”.



Meal Plan	Shopping List
Monday: Spaghetti Carbonara	
Tuesday: Spaghetti Carbonara	
Wednesday: Spaghetti Carbonara	
Thursday: Spaghetti Carbonara	
Friday: Spaghetti Carbonara	
Saturday: Spaghetti Carbonara	
Sunday: Spaghetti Carbonara	
	Spaghetti
	Parmesan
	Bacon
	Eggs

Generate

2.2. Basic Functionality (10 Points)

Modify the `MainWindowViewModel` and add Bindings to the `MainWindow.axaml` so that the program fulfils the following requirements:

1. Left ListBox (Meal Plan)

- Displays a recipe for each weekday.
- Use the provided `WeekDayItem` class to represent each item.
- Use the provided `MealPlanner` instance to generate the list.

2. Right ListBox (Shopping List)

- Displays the shopping list, that corresponds to the current meal plan.
- This list is generated based on the current meal plan using the `ShoppingListGenerator` Instance.

3. Generate Button

- When clicked, it should:
 - Regenerate the list of recipes.
 - Update the shopping list accordingly.

On startup, the program should already generate a Meal plan for one week and display it. All needed instances from Problem 1 are already available in the `MainWindowViewModel`.

2.3. Advanced Functionality (10 Points)

Further modify the `MainWindowViewModel.cs` and update the bindings in `MainWindow.axaml` so that:

When a user **selects a recipe** (i.e. clicks on an item) in the **left ListBox** (which displays the meal plan) the **corresponding ingredients** for that recipe are automatically **highlighted (selected)** in the **right ListBox** (which displays the shopping list).

Hint:

When using the `[ObservableProperty]` Attribute from the MVVM Toolkit, a partial method named `On{PropertyName}Changed` is automatically generated.

The ViewModel already contains a property for the selected Recipe and the corresponding function, to help you react when the selected recipe changes:

```
[ObservableProperty]
private WeekDayItem selectedRecipe;

partial void OnSelectedRecipeChanged(WeekDayItem value)
{ // Do Something here }
```

Problem 3 – Asynchronous Programming (20 Points)

In the folder /Problem_3_Async/Counter/, you will find an Avalonia application called **Counter**.

The UI includes a TextBlock that displays a number and three buttons:

- **Start** – begins automatically increasing the number.
- **Pause** – pauses the counting.
- **Reset** – resets the number to zero.

Only add the following file for this problem to your submission folder:

Problem_3_Async/Counter/ViewModels/AutoCounterViewModel.cs

Modify the AutoCounterViewModel so that it meets the following requirements:

- **Start Button:** When pressed, the number increases by **1 every 100 milliseconds**.
(10 Points)
- **Pause Button:** When pressed, the counter **stops**. If the Start button is pressed again, it should **continue from the last number**.
(4 Points)
- **Reset Button:** When pressed, the counter **resets to zero**.
(4 Points)
- **Thread Safety:** Ensure that **all UI updates** are performed on the **UI thread**.
(2 Points)

Problem 4 – LINQ (25 Points)

You are given a JSON file (Problem_4_LINQ/data/recipes.json) that contains a database on Recipes similar to the one from Problem 1.

Add your Program.cs file containing the queries as Problem_4_Program.cs and the resulting json file as Problem_4_Query_Results.json to your submission folder.

4.1. Run Queries (20 Points)

Run the following five LINQ queries on the list of recipes:

1. **Vegetarian Recipes**
Return all recipes marked as vegetarian.
(5 Points)
2. **No Dietary Restrictions**
Return all recipes that do **not** have any dietary restrictions.
(5 Points)
3. **Sorted by Ingredient Count**
Return all recipes sorted by the number of ingredients, with the recipes using the **most ingredients at the top**.
(5 Points)
4. **Recipes with more Ingredients than the Average**
Return all recipes that use more ingredients than the average number of ingredients across the entire recipe list.
(5 Points)

4.2. Save JSON (5 Points)

Save the results of all four queries into a single JSON file named and add it to your submission folder.

The resulting file should contain a structure like this:

```
{  
  "vegetarianRecipes": [ ... ],  
  "noDietaryRestrictions": [ ... ],  
  "sortedByIngredientCount": [ ... ],  
  "aboveAverageIngredients": [ ... ]  
}
```